

GUIA DE C++

Sintaxe moderna, RAII, templates e STL

C++20 com notas de C++23

Sintaxe + exemplos + biblioteca padrao + boas praticas

Edicao de estudo | Setembro de 2026

Como usar este livro

Este livro-resumo foi feito para estudo e consulta. Leia em ordem na primeira passagem e depois use cada capítulo como referencia. Ele cobre o núcleo, as famílias de funções e os componentes padrão; APIs de sistema e bibliotecas de terceiros ficam fora do escopo.

Legenda mental

- Sintaxe e a forma válida de escrever.
- Semântica e o significado e o momento de execução.
- Biblioteca padrão adiciona tipos e funções portáveis.
- Comportamento indefinido significa que o padrão não promete resultado.

Escopo: abrangente como livro-resumo, mas não substitui a especificação normativa nem a documentação exata do compilador.

Mapa do livro

- 1. Como C++ funciona
- 2. Tipos, inicialização e dedução
- 3. Expressões e controle
- 4. Funções e lambdas
- 5. Classes e encapsulamento
- 6. RAII e funções especiais
- 7. Herança e polimorfismo
- 8. Templates e conceitos
- 9. Movimento e categorias de valor
- 10. Containers e iteradores
- 11. Algoritmos, ranges e views
- 12. Erros e exceções
- 13. Memória e smart pointers
- 14. Entrada, saída e arquivos
- 15. Concorrência
- 16. Biblioteca padrão: mapa
- 17. Biblioteca: componentes essenciais
- 18. Módulos e compilação
- 19. Compile-time e segurança
- 20. Checklist final

1. Como C++ funciona

C++ é compilada e multiparadigma. O sistema de tipos, o tempo de vida e a biblioteca padrão permitem abstrações de alto nível sem custo obrigatório.

```
#include <iostream>
int main() {
    std::cout <<< "Ola, C++!\n";
}
```

- std é o namespace da biblioteca padrão.
- Objetos são construídos e destruídos em momentos definidos.
- Cabeçalhos e fontes formam unidades de tradução ligadas ao final.
- C++ não é apenas C com classes; possui regras próprias.

```
c++ -std=c++20 -Wall -Wextra -Wpedantic programa.cpp -o programa
```

2. Tipos, inicializacao e deducao

C++ acrescenta referencias, tipos de classe, nullptr e deducao controlada aos tipos fundamentais.

```
int quantidade{10};
const double taxa{0.15};
auto total = quantidade * taxa;
std::int64_t id{42};
int* p = nullptr;
decltype(total) copia{};
```

Forma	Significado
T x;	padrao; fundamental local pode ficar indeterminado
T x{};	lista; evita estreitamento e normalmente zera
auto x = expr;	deduz tipo do valor
const T& r=x;	referencia somente leitura
T&& r=expr;	referencia rvalue

Em resumo: Use chaves para detectar perda. Use auto quando o tipo continuar claro.

3. Expressões e controle

Operadores seguem precedencia semelhante a C, com sobrecarga para tipos de usuario.

Grupo	Operadores e regra
Aritmetica	+ - * / %; divisao inteira descarta fracao
Comparacao	== != < <= > >=
Logica	&& !; curto-circuito em && e
Bits	& ^ ~ << >> em inteiros
Atribuicao	= += -= *= /= %= e formas de bits
Acesso	[] () . ->
Outros	++ -- ?: , sizeof; use parenteses quando houver duvida

```
if (auto it = mapa.find(chave); it != mapa.end())
    usar(it->second);
```

```
for (const auto& item : colecao) processar(item);
```

```
switch (estado) {
case Estado::pronto: iniciar(); break;
case Estado::erro: recuperar(); break;
default: break;
}
```

- if/switch podem conter inicializacao.
- for de intervalo usa begin/end.
- RAII torna saidas antecipadas seguras.
- Sobrecarga deve preservar a intuicao do operador.

4. Funcoes e lambdas

Funcoes podem ser sobrecarregadas, ter parametros padrao e retorno deduzido. Lambdas criam objetos chamaveis.

```
int somar(int a,int b) { return a+b; }  
double somar(double a,double b) { return a+b; }  
  
[[nodiscard]] std::optional<int> dividir(int a,int b) {  
    if (!b) return std::nullopt;  
    return a/b;  
}  
auto dobro = [](int x) noexcept { return 2*x; };
```

- Valor para tipos pequenos/copia; const& para observar; & para alterar; && para mover.
- Parametro padrao aparece normalmente na declaracao.
- Lambda captura por valor [=], referencia [&] ou nomes.
- std::function apaga o tipo, com possivel custo; template preserva.

5. Classes e encapsulamento

Classes combinam invariantes, dados e operacoes. struct e class diferem somente nos acessos padrao.

```
class Conta {  
public:  
    explicit Conta(double s=0) : saldo_{s} {}  
    void depositar(double v) { if (v>0) saldo_+=v; }  
    [[nodiscard]] double saldo() const noexcept { return saldo_; }  
private:  
    double saldo_{};  
};
```

- public forma a interface; private protege invariantes.
- Construtor estabelece estado valido; destrutor finaliza.
- Membro const nao altera estado observavel.
- static pertence a classe.
- explicit bloqueia conversoes implicitas indesejadas.
- friend concede acesso pontual.

6. RAI e funcoes especiais

RAI associa um recurso a um objeto. Saida de escopo, return e execucao executam destruidores.

Funcao especial	Papel
Construtor padrao	cria sem argumentos
Destrutor	finaliza e libera
Construtor/atribuicao de copia	duplica valor
Construtor/atribuicao de movimento	transfere recursos

```
class Arquivo {  
    std::FILE* f_{};  
public:  
    explicit Arquivo(const char* n) : f_{std::fopen(n,"rb")} {  
        if (!f_) throw std::runtime_error("falha");  
    }  
    ~Arquivo() { std::fclose(f_); }  
    Arquivo(const Arquivo&) = delete;  
    Arquivo& operator=(const Arquivo&) = delete;  
};
```

Em resumo: Regra de zero: use membros que gerenciam recursos. Se gerenciar manualmente, analise destrutor, copia e movimento: regra de cinco.

7. Heranca e polimorfismo

Heranca publica modela e-um. virtual permite despacho dinamico por referencia ou ponteiro para a base.

```
struct Forma {  
    virtual ~Forma() = default;  
    virtual double area() const = 0;  
};  
struct Circulo final : Forma {  
    double r;  
    explicit Circulo(double x): r{x} {}  
    double area() const override { return 3.141592653589793*r*r; }  
};
```

- Base polimorfica usada para delecao precisa de destrutor virtual.
- override verifica assinatura; final impede extensao.
- Evite slicing: passe por referencia ou ponteiro.
- dynamic_cast verifica conversao polimorfica.
- Prefira composicao sem substituibilidade real.

8. Templates e conceitos

Templates geram código para tipos/valores. Conceitos expressam requisitos e melhoram diagnósticos.

```
template<class T>
T maximo(const T& a, const T& b) { return a < b ? b : a; }
```

```
template<std::totally_ordered T>
T limitar(T x, T lo, T hi) { return std::clamp(x, lo, hi); }
```

```
template<class T>
concept Somavel = requires(T a, T b) { a+b; };
```

- Definições de templates normalmente ficam em cabeçalhos.
- if constexpr descarta ramos em compilação.
- Especialização e overload tratam variações.
- Variadic templates e fold expressions operam em pacotes.

9. Movimento e categorias de valor

`std::move` apenas converte para rvalue; a operacao seguinte decide se transfere.

```
std::string a="dados";  
std::string b=std::move(a); // a valida, conteudo nao especificado
```

```
template<class F,class... A>  
decltype(auto) chamar(F&& f,A&&... a) {  
    return std::invoke(std::forward<F>(f),  
                       std::forward<A>(a)...);  
}
```

- lvalue tem identidade persistente; rvalue costuma ser temporario/movivel.
- Objeto movido permanece valido e destrutivel.
- Copy elision constroi diretamente no destino.
- `std::forward` preserva a categoria em templates.

10. Containers e iteradores

Escolha o container pela operacao dominante, complexidade e localidade.

Tipo	Uso
array	fixo e contiguo
vector	dinamico e contiguo; escolha padrao
deque	duas pontas
list / forward_list	nos encadeados
map / set	arvore ordenada
unordered_map / set	hash sem ordem
stack / queue / priority_queue	adaptadores
span	visao contigua nao proprietaria
string / string_view	texto proprietario / visao

Em resumo: Invalidacao depende do container e operacao. span e string_view nao prolongam a vida dos dados.

11. Algoritmos, ranges e views

Algoritmos separam operacao do container. Ranges aceitam o intervalo; views compoem transformacoes preguiçosas.

```
std::vector<int> v{4,1,3,2};
std::ranges::sort(v);
auto pos=std::ranges::find(v,3);
int soma=std::accumulate(v.begin(),v.end(),0);

auto pares=v | std::views::filter([](int x){return x%2==0;})
             | std::views::transform([](int x){return x*x;});
```

Familia	Exemplos
Busca	find, find_if, binary_search, lower_bound
Ordenacao	sort, stable_sort, partial_sort, nth_element
Transferencia	copy, move, transform, fill, generate
Particao	partition, stable_partition
Conjuntos	set_union, set_intersection, includes
Heap	make_heap, push_heap, pop_heap
Numericos	accumulate, reduce, inner_product, iota
Predicados	all_of, any_of, none_of, equal
Permutacao	reverse, rotate, shuffle, next_permutation

12. Erros e excecoes

Excecoes representam falhas que impedem a operacao. Tipos de resultado modelam ausencia ou alternativas.

```
try { carregar(); }  
catch (const std::filesystem::filesystem_error& e) {  
    std::cerr <<< e.what() <<< '\n';  
}  
catch (const std::exception& e) {  
    std::cerr <<< e.what() <<< '\n';  
}
```

- Lance por valor; capture por referencia const.
- Destrutores devem ser noexcept.
- optional representa talvez; variant alternativas; expected<T,E> sucesso/erro em C++23.
- error_code atende APIs sem excecoes.
- Excecao escapando de noexcept chama terminate.

13. Memoria e smart pointers

Prefira valores e containers. Smart pointers tornam propriedade explicita.

Tipo	Semantica
unique_ptr	proprietario unico
shared_ptr	propriedade compartilhada por contagem
weak_ptr	observador expiravel; quebra ciclos
make_unique/make_shared	construcao segura
span	visao sem propriedade

```
auto p=std::make_unique<Conta>(100);  
auto s=std::make_shared<Conta>(50);  
std::weak_ptr<Conta> w=s;  
if (auto vivo=w.lock()) vivo->depositar(10);
```

Em resumo: Comece com valor ou unique_ptr. Use shared_ptr somente quando o dominio realmente compartilhar propriedade.

14. Entrada, saida e arquivos

Streams tratam E/S; filesystem trata caminhos e diretorios; format oferece formatacao segura por tipo.

```
std::ifstream in{"dados.txt"};
if (!in) throw std::runtime_error("nao abriu");
std::string linha;
while (std::getline(in,linha)) std::cout <<< linha <<< '\n';

std::filesystem::path p="saida";
p /= "log.txt";
std::cout <<< std::format("itens: {}",42);
```

- iostream: cin, cout, cerr, clog.
- fstream: ifstream, ofstream, fstream.
- sstream: fluxo em memoria.
- format: format e format_to; print em C++23.
- filesystem: path, iteracao, copia, remocao e metadados.

15. Concorrencia

Threads compartilham memoria; sincronizacao preserva invariantes e estabelece ordem.

```
std::mutex m;  
int total=0;  
void somar(int x) {  
    std::scoped_lock lock{m};  
    total += x;  
}  
std::jthread a{somar,2};  
std::jthread b{somar,3};
```

Componente	Uso
thread / jthread	execucao; jthread junta e permite parada
mutex / scoped_lock	exclusao mutua RAI
condition_variable	espera por estado
atomic	operacoes indivisiveis
future / promise / async	resultado assincrono
latch / barrier / semaphore	coordenacao C++20
stop_token	cancelamento cooperativo

Em resumo: Data race e comportamento indefinido. Minimize compartilhamento e nunca gerencie mutex manualmente quando um guard RAI resolve.

16. Biblioteca padrao: mapa

A biblioteca evolui por versao. Este mapa cobre os componentes de uso geral mais importantes.

Area	Cabecalhos/componentes
Fundamentos	cstdint, cstdint, limits, type_traits, utility, tuple, compare
Memoria	memory, memory_resource, new
Resultados/erros	exception, stdexcept, system_error, optional, variant, any, expected
Texto	string, string_view, charconv, format, regex, locale
Containers	array, vector, deque, list, map, set, unordered_*, stack, queue, span
Iteracao	iterator, ranges
Algoritmos	algorithm, numeric, execution
E/S	iostream, iomanip, fstream, sstream, syncstream, print
Tempo	chrono, ratio, ctime
Matematica	cmath, complex, valarray, numbers, random, bit
Concorrencia	thread, mutex, condition_variable, future, atomic, semaphore, latch, barrier
Sistema	filesystem, source_location
Compilacao	concepts, type_traits, bitset
Compatibilidade C	cstdio, cstdlib, cstring, cmath e outros c...

17. Biblioteca: componentes essenciais

Memorize familias, nao cada assinatura. Consulte contratos completos de complexidade, invalidacao e execucoes.

Familia	Nomes-chave
Utilidades	move, forward, swap, exchange, pair, tuple, apply, invoke
Tipos	is_same, is_integral, remove_reference, decay, common_type
Memoria	unique/shared/weak_ptr, allocator, construct_at, uninitialized_*
Strings	stoi, to_string, from_chars, to_chars, string_view
Ranges	begin, end, size; views::filter, transform, take, drop, iota
Aleatorio	random_device, mt19937, uniform_int_distribution
Tempo	duration, time_point, steady_clock, system_clock
Bits	bit_cast, popcount, countl_zero, rotl
Numericos	gcd, lcm, midpoint, lerp, numbers::pi
Diagnostico	assert, static_assert, source_location, exception_ptr
Filesystem	path, exists, directory_iterator, create_directories, copy, remove
Regex	regex, regex_match, regex_search, regex_replace

18. Modulos e compilacao

Cabecalhos tradicionais exigem protecao. Modulos C++20 criam fronteira explicita, mas o suporte de ferramentas varia.

```
// matematica.hpp
#pragma once
namespace app { int somar(int,int); }

// matematica.cpp
#include "matematica.hpp"
int app::somar(int a,int b) { return a+b; }

// modulo
export module app.math;
export int dobro(int x) { return 2*x; }
```

- Nao use using namespace std; em cabecalhos.
- ODR exige definicoes coerentes; inline permite certas definicoes em cabecalho.
- Templates precisam da definicao visivel.
- Use sistema de build e fixe versao, opcoes e dependencias.

19. Compile-time e seguranca

`constexpr` permite avaliacao em compilacao; `constexpr` exige. Abstracoes modernas ajudam a evitar comportamento indefinido.

```
constexpr int quadrado(int x) { return x*x; }  
static_assert(quadrado(5)==25);
```

```
constexpr unsigned id(unsigned x) { return x*2654435761u; }
```

- `constexpr` pode executar em runtime; depende do contexto.
- `constexpr` garante inicializacao estatica.
- Evite referencias/views pendentes e indices fora dos limites.
- Nao estoure signed, use shift invalido ou objeto indeterminado.
- Nao misture `new/delete`, `new[]/delete[]` ou familias.
- Use warnings, sanitizers, testes e analise estatica.

20. Checklist final

C++ moderno modela propriedade e estados nos tipos.

Situacao	Preferencia
Sequencia	vector
Texto observado	string_view com vida garantida
Propriedade	valor ou unique_ptr
Talvez	optional
Alternativas	variant + visit
Erro rico	excecao ou expected
Bloqueio	scoped_lock
Algoritmo	std::ranges
Conversao	static_cast; evite cast C
Interface	const, nodiscard, noexcept quando verdadeiro

Em resumo: Quando propriedade e tempo de vida aparecem nos tipos, estados invalidos ficam mais dificeis de representar.

Proximos passos

Reescreva os exemplos, preveja resultados antes de executar e construa programas pequenos completos. Quando um contrato exato importar, consulte uma referencia atualizada da versao escolhida.

- Revisar tipos e expressoes.
- Praticar controle e funcoes.
- Dominar memoria e tempo de vida.
- Aplicar biblioteca padrao.
- Separar um projeto em arquivos.
- Adicionar testes, avisos e analisadores.